

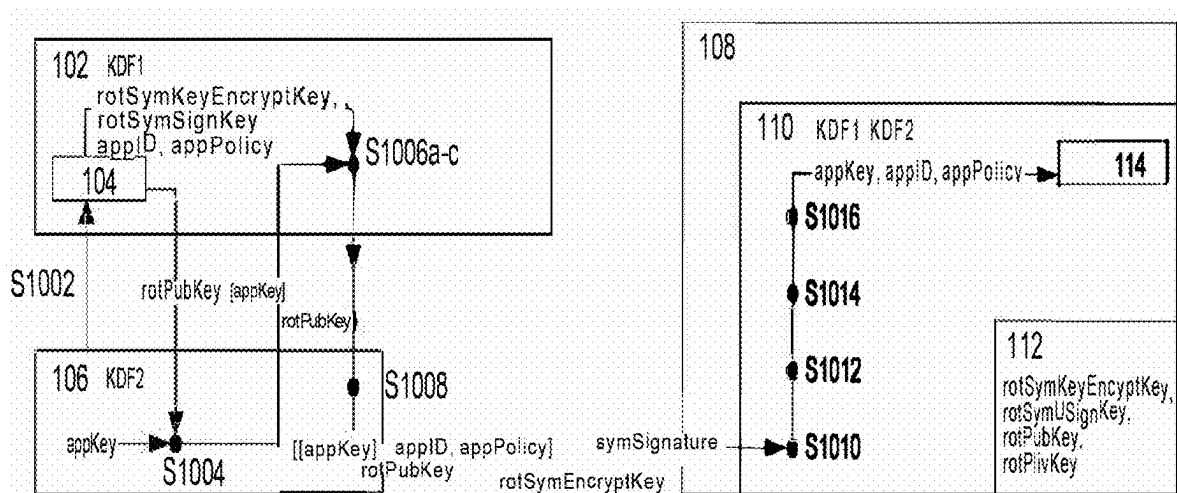


Fig. 1

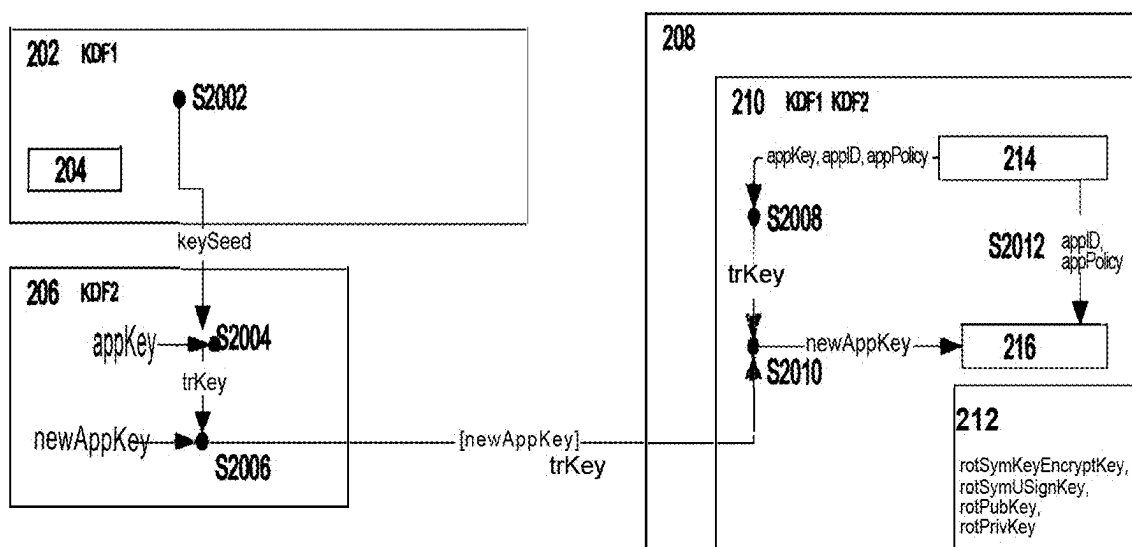


Fig. 2

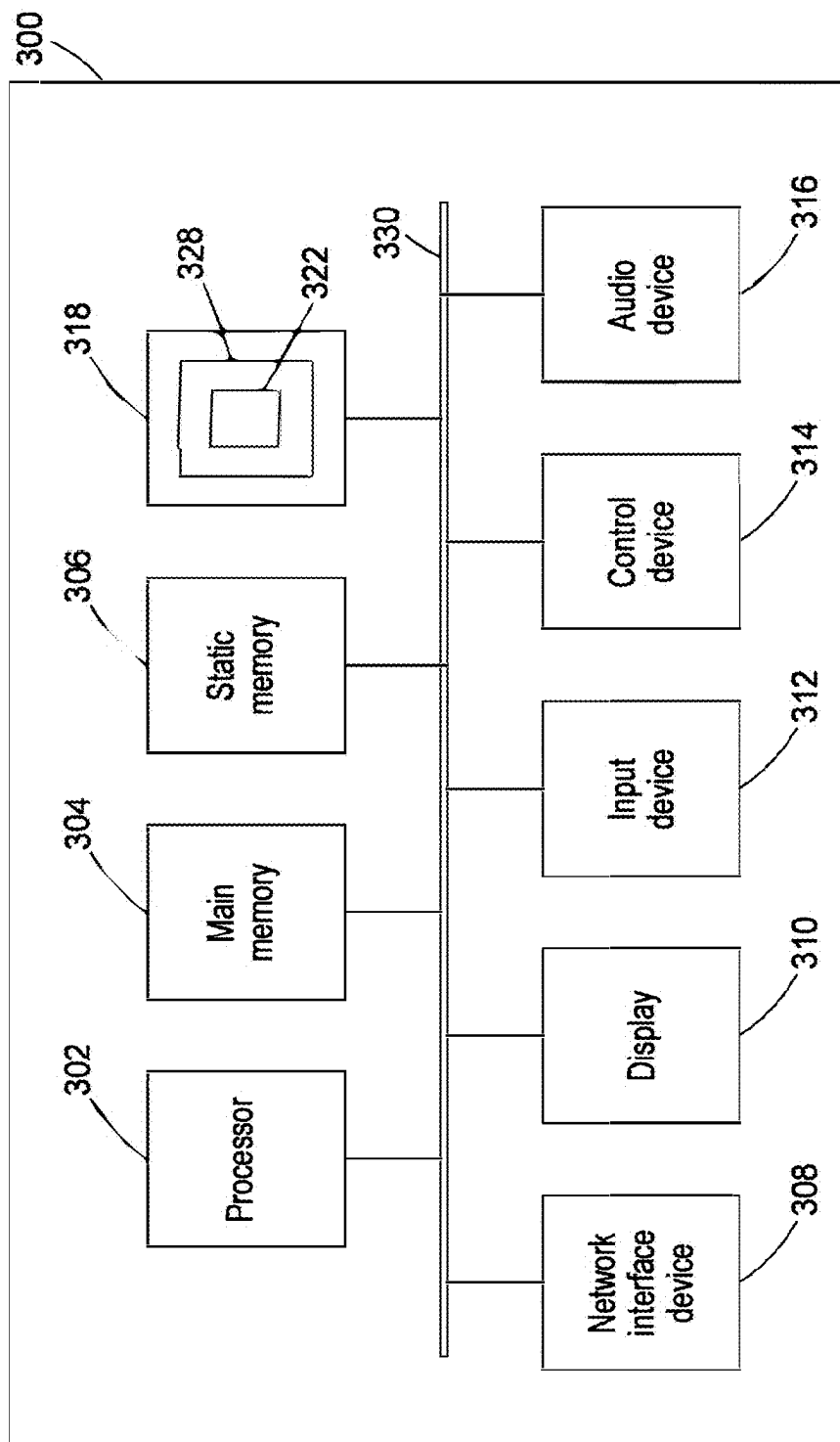


Fig. 3

SECURE INSTALLATION OF APPLICATION KEYS

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application is a continuation of U.S. patent application Ser. No. 18/542,551, filed Dec. 15, 2023, which is a continuation of U.S. patent application Ser. No. 17/941,909, filed Sep. 9, 2022, which is a continuation of U.S. patent application Ser. No. 16/958,104, filed Jun. 25, 2020 (each of which are incorporated herein by reference in their entirety), which is a National Stage of PCT/EP2018/086576 filed Dec. 21, 2018, and claims priority to European Patent Application No. 17306983.2 filed Dec. 29, 2017.

FIELD

[0002] The present disclosure relates to the installation of keys associated with an application on a remote device, for managing execution of the application on the remote device.

BACKGROUND

[0003] Remote devices capable of executing one or more applications or processes relating to an application, require secure methods of data transfer, updates of application policies and/or associated keys from an application owner in order for the remote device to access the application or execute processes associated with it. The application is usually owned and managed by or from an application owner device. Commonly, installation of application keys and secure data transfer can be achieved by symmetric and asymmetric key encryption methods.

[0004] Although secure transfer can be achieved by these techniques, this often leads to large bandwidth use each time the remote device requires or acquires new application keys, policies or other application specific credentials from the application owner device, as this requires both symmetric and asymmetric methods to ensure secure transfer.

[0005] There are restrictions from the owner of the remote device, which may be different from those that may be the application owner. The owner of an application embedded in a remote device will want to secure the data generated or received by his application, so that confidentiality and integrity of the data cannot be compromised. There is thus a need for a solution that allows the owner of the remote device, typically a Network Operator, to assert specific policy(ies) to control the data generated or received by an application, with limited communication bandwidth and power constraints. There is thus a need to provide the owner of a remote device, a solution to limit an application to generate or receive data in a specific physical location, for example.

BRIEF DESCRIPTION OF THE DRAWINGS

[0006] Various embodiments are now described by way of example for the purpose of explanation and illustration, with reference to the accompanying drawings in which:

[0007] FIG. 1 is a schematic depicting a first embodiment of the disclosure for the installation of an application policy on a remote device for execution of the respective application;

[0008] FIG. 2 is a schematic depicting a second embodiment of the disclosure for the installation of further application keys for further operation of respective application on the device; and

[0009] FIG. 3 is an example implementation of a device according to the described embodiments.

DETAILED DESCRIPTION

[0010] In overview, methods and devices in accordance with the present disclosure provide secure installation of application keys associated with an application from an application owner (also referred to as an application owner device) onto a remote device in a different physical location to the owner device. The methods and devices may improve or optimise bandwidth usage during the installation. Data security is particularly important when a remote devices or network is communicatively coupled with one or more application owners. Hence this requires a secure method by which keys, policies and identifiers, and any other measures of data security cannot be by-passed.

[0011] The use of symmetric and asymmetric keys and associated encryption methods although secure, use a lot of network and processing resources, especially network bandwidth of a wireless communication network during installation. For example, considering ECIES encryption using a classical MAC signature, the extra size per message is around 80 bytes. Additionally, since application keys are commonly renewed, sometimes on a regular basis, similar transfer methods are used to ensure secure transfer of new application keys and consequently lead to large bandwidth costs and strains on the network each time new application keys are required to be installed. In addition, an application policy dictated by the device owner, which may be a set of executable code or instructions dictating one or more features of or limitations concerning the application, may also need to be sent each time new keys are sent.

[0012] The present disclosure, in a first embodiment, proposes methods, devices and systems for transfer a first application key, also known as an application root key in some cases, to a remote device.

[0013] An associated policy for the application, which is preferably defined by the remote device owner for the respective application, rather than the application owner, may also be associated with the root application key transfer. Along with the application root key, an identifier of the respective application may also be also provided or transferred to the remote device, said identifier associated with the respected application to be executed on the remote device, through secure key exchange protocols, preferably with both symmetric and asymmetric encryption methods.

[0014] The application policy is defined, and may be stored or processed at a key management system (KMS) in one aspect. The KMS may preferably be controlled by the remote device owner. The remote device is configured to, once the remote device receives the application policy, to decrypt the first application key for the application/application owner, the identifier, and its associated policy and then store the first application key, application policy and application identifier for further application usage.

[0015] Typically, application owners obtain, store or generate further application keys for transfer to the remote device for further or subsequent usage of the application at the remote device. In most cases application keys are symmetric keys, and they may also be for an instant use, i.e.

a sole use until the end of an application session or to execute a single process associated with the application. In some cases, the key may be valid for a prescribed length of time, or a range of timings depending on the process executed by the remote device.

[0016] However, the use of only symmetric keys is not as secure as a combination of symmetric and asymmetric keys for specifying of application policies on a remote device. This may be problematic when the remote device owner wants to control usage of the application to prevent unauthorised access or execution rights for the processes associated with the application on the device. Thus, both types of keys are generally envisaged to be used, for example when sending an application policy for installation. There may be some highly secure applications, i.e. such as a patch of payment software applications or applications using sensitive data, where both symmetric and asymmetric keys are always sent regardless of whether or not the policy is re-installed each time.

[0017] The present disclosure further proposes in a second embodiment methods, devices and systems for secure installation of subsequent or further application keys of an application on a device that already has a respective policy associated with the application installed in the device. This is so that for secure subsequent or further use of the application, at the remote device, only the application keys, which are typically symmetric keys, need to be installed.

[0018] This technique involves generating a key seed, preferably in the KMS, which may be controlled by the remote device owner, and for example an operator, with an application policy and/or identifier using a predefined first key derivation function. This key seed is sent to the application owner which then generates a transport key at the application owner with the key seed and an application root key or first application key using a further key derivation function. The application root key is available in the remote device, for example installed as described or otherwise.

[0019] This transport key is used to encrypt a new or each subsequent application key to be installed at the device. Thus, now it is just this new or subsequent or further application key that needs to be transferred to be installed at the device, which thereby occupies significantly less bandwidth when compared to requiring symmetric, asymmetric keys and a policy in some cases, each time the device runs the application

[0020] Thus, there is no means for obtaining, decrypting or using the new or further application key from the application owner without first having access to this transport key, which is generated on the basis of the application policy, and preferably an application identifier. Hence, the application owner can securely transfer the new application keys for subsequent application use to the remote device, utilising less bandwidth, as the new or further application key is typically sized at 16 bytes or so.

[0021] The remote device is configured to store or access the above mentioned key derivation functions and has the first or root application key, policy and identifier. One or more computing units associated with the device are configured to regenerate the key seed and therefore can obtain the transport key required to decrypt the received new application key. The new application key may then replace the previous application key and inherit the application policy and identifier, hence further transfer of policies and identifiers are not required. The remote device may store the

new application key for further application usage, in one aspect. Otherwise, the process briefly set out about in respect of the second embodiment may be repeated with another new key replacing the present new key.

[0022] Although, the initial transfer of the root or first application key, the respective application identifier and preferably its associated policy may potentially take place at a high bandwidth and associated cost; once they have been securely installed at the remote device, further or new application keys can be installed using significantly low bandwidth and/or associated resources. The new application keys are encrypted with a transport key which is generated from a key derivation function, which may be at a lower processing and network resource cost, as symmetric key encryption methods can be employed for this, which are less data intensive than asymmetric key encryption methods. Hence all further, new or subsequent application keys can be securely transferred using on symmetric keys.

[0023] Some specific embodiments and components associated with each embodiment are now described by way of illustration with reference to the accompanying drawings, in which like reference numerals refer to like features.

[0024] Embodiments of the present disclosure includes a method of preparing one or more application keys owned by application owners for installing in a remote device, each application preferably having a specific application policy managed by the device owner which never knows the application keys. This can be implemented by using a KMS communicatively coupled to the application owner, whereby communication is established via a wired or wireless communication network, and data may be transferred by well-known data transfer protocols.

[0025] The KMS is in one aspect responsible for processing application keys and/or generating key seeds for secure transfer of further application keys. The KMS is also configured to include a memory or storage area for obtaining and storing one or more application identifiers and respective policies which relate to the application owner. The application identifiers allow the remote device to associate the application keys to its application owners and the application policies that regulate the data transfer between remote device and application owners. One or more keys may also be stored to process the application keys from the application owner for secure transfer to the remote device. The KMS further may be configured to have at least one processing module for holding a key generation function for generating a key seed for secure transfer of further application keys from application owners to a remote device.

[0026] A first aspect by which a KMS can be implemented for secure transfer of an application key from an application owner to a remote device is shown in FIG. 1, which relates to a first embodiment of the present disclosure for installing the root or first application key with the associated policy. The KMS 102 is communicatively coupled to an application owner 106, and the application owner is communicatively coupled to a remote device 108.

[0027] The KMS 102 is configured to include a database 104 which stores respective application policies appPolicy and identifiers appID that can be associated to each application at the application owner 106. The KMS 102 is also configured to obtain and store an asymmetric key pair associated with the device 108, which includes device public key corresponding to a device private key stored on and specific to the remote device 108, indicated herein as rot-

PubKey, and device symmetric keys for signature processing and use in a key exchange process, indicated herein as rotSymSignKey and rotSymKeyEncryptKey, respectively. The KMS 102 also holds a predefined key derivation function KDF1 for key seed generation, which is required for secure transfer of one or more further application keys. This may be within at least one processing module in the KMS102.

[0028] Step S1002 involves a request for the device public key rotPubKey from the application owner 106 sent to the KMS 102. The KMS 102 then sends the rotPubKey to the application owner 106. This is facilitated using one or more transceiver modules in the KMS102.

[0029] In step S1006a, the KMS 102 receives from the application owner 106 the application key, referred to herein as appKey, which is encrypted by the rotPubKey, indicated by [appKey] rotPubKey.

[0030] In step S1006b, the KMS 102 further processes the received encrypted application key, [appKey] rotPubKey, by adding an appID and appPolicy from the database 104, or associating the appID and appPolicy with the received encrypted key.

[0031] Step S1006c further involves encrypting said processed key with the device symmetric encryption key, indicated as rotSymmKeyEncryptKey from the database 104. Processing the received encrypted application key may further involve being signed with a symmetric device signature key.

[0032] In step S1008, once the appID and appPolicy have been associated or added, and encryption and signature generation is complete, the fully processed application key, indicated by [[appKey]rotPubKey, appID, appPolicy]rotSymmEncryptKey symSignature, is sent to the application owner 106 for further transfer to the remote device 108. Alternatively, the fully processed application key may be sent directly to the remote device 108 by the KMS 102.

[0033] A second aspect by which the KMS can be implemented for secure transfer of an application key from an application owner to a remote device is shown in FIG. 2, which relates to a second embodiment of the present disclosure for installing subsequent application keys.

[0034] FIG. 2. shows a method for which a KMS 202, can process further application keys, indicated as newAppKey from an application owner 206 to a remote device 208. For the purposes of the suggested implementation of the present disclosure, the KMS 202 contains the same configurations and components as described above in the first aspect, including transceiver ports and one or more processing modules.

[0035] In step S2002, the KMS 202 generates a key seed, indicated as keySeed from a predefined key derivation function, indicated as KDF1 in a processing module, with the appID and preferably the appPolicy. This is then sent to the application owner 206 for further processing.

[0036] A third aspect of the disclosure involves a method of installing a first or root application key and its associated policy from an application owner onto a remote device. This can be implemented by communicatively coupling the application owner to a KMS and remote device, whereby the KMS is configured to act as an application key processor, which ensures secure transfer of the application keys, before the application can send the application keys to the remote device.

[0037] One example for which a first or root application key can be installed securely from an application owner onto a remote device is shown in FIG. 1, which relates to a first embodiment of the present disclosure for installing the root or first key with the associated policy.

[0038] For purposes of the suggested implementation of the present disclosure, KMS 102 in FIG. 1 includes the same components and configurations as described above in the first aspect. The application owner 106 is configured to store an appKey, which is also referred to as a first or root application key, as well as further application keys, such as newAppkey, for installing on the remote device 108, and holds a further key derivation function KDF2, which is configured to generate transport keys for secure transfer of further application keys when implemented.

[0039] In step S1002, the application owner 106 requests the rotPubKey from the KMS 102.

[0040] In step S1004, the application owner 106 receives the rotPubKey from the KMS 102 and uses it to encrypt the application key appKey, which can only be decrypted with a device private key, indicated herein as rotPrivKey and therefore ensures secure transfer of the appKey to the KMS 102.

[0041] In step S1008, the application owner 106 receives the appKey which has been processed by the KMS 102 as described above, given by [[appKey]rotPubKey, appID, appPolicy]rotSymmEncryptKey symSignature, and this is further sent from the application owner 106 to the remote device 108 for secure installation.

[0042] A fourth aspect of the disclosure is seen in FIG. 2., which relates to a second embodiment of the present disclosure for installing subsequent application keys, involves a method of installing subsequent or further application keys from an application owner 206 onto a remote device 208. The subsequent keys may be stored in the application owner 206 in a storage module in the form of a key list or key ladder for each subsequent use, as necessary.

[0043] For the purposes of the suggested implementation of this present disclosure, the KMS 202 and application owner 206 contain the same components and configurations as described above for the second and third aspects.

[0044] In step S2002, the application owner 206 receives the keySeed generated in the KMS 202. A transport key, indicated herein as trKey is generated by the application owner 206 using the keySeed and the appKey that was previously transferred to the remote device 208 with a further key derivation function KDF2.

[0045] In step S2006, the application owner 206 uses this trKey to encrypt the newAppKey, and the encrypted key, indicated by [newAppKey] trKey, is then sent to the remote device 208 for installation.

[0046] A fifth aspect of the present disclosure relates to a remote device which can execute an application using one or more application keys. The remote device is communicatively coupled to the application owner, whereby the application keys, policies and identifiers can be securely transferred from the application owner. The remote device is further configured to store its public key from its public and private key pair, symmetric keys and predefined key derivation functions in the device's memory.

[0047] One example by which the remote device can be implemented for processing and storage of application keys

is shown in FIG. 1, which relates to a first embodiment of the present disclosure for installing the root or first key with the associated policy.

[0048] For the purposes of the suggested implementation of the present disclosure, the KMS 102 and application owner 106 in FIG. 1, contain the same components and configurations as described above in relation to the first and third aspect. The remote device 108 includes a trusted processing module, which may be an embedded Root of Trust 110, indicated as ROT 110 for the device 108. This is shown as RoT 110, which may also include a one-time programmable (OTP) storage 112 and a memory storage 114.

[0049] The OTP storage 112 stores rotSymUSignKey, eotSymEncryptKey, rotPubKey and rotPrivKey for decryption of incoming application keys. The RoT 110 is also configured to implement one or more key derivation functions KDF1, KDF2 required to generate the keySeed and trKey described above for further application key decryption.

[0050] In step S1010, the remote device 108 receives the encrypted application key, indicated by [[appKey]rot-PubKeyappID,appPolicy]rotSymEncryptKey symSignature, from the application owner 106 (or the KMS 102) and checks the signature with rotSymUSignKey.

[0051] In step S1012, the remote device 108 decrypts this key with rotSymEncryptKey.

[0052] In step S1014, the remote device 108 verifies the appPolicy and associated appID.

[0053] In step S1016, the remote device 108 further decrypts this key with the rotPrivKey and stores the decrypted application key, policy and identifier in its memory 114, for further use, such as executing the application based on the associated policy.

[0054] In a sixth aspect of the disclosure; having gained access to the application key, policy and identifier obtain as obtained in the fifth aspect above, the remote device 108 can now receive further application keys making efficient use of available bandwidth.

[0055] This is illustrated in FIG. 2, associated with the second embodiment of the present disclosure for installing subsequent application keys. For the purpose of the suggested implementation of the disclosure, the KMS 202, application owner 206 and remote device 208 include the same components as set out in relation to the first to fifth aspects. The remote device 208 may include a further memory 216 in the RoT 210.

[0056] In step S2008, the RoT 210 receives the appKey, appID and appPolicy from its memory 214 and generates the keySeed and the trKey through key derivation functions KDF1, KDF2 as described in the second and the fourth aspect.

[0057] In step S2010, the RoT 210 receives a newAppKey that has been encrypted with the trKey from the application owner 206, given by [newAppKey] trKey. The generated trKey is used to decrypt this new application key, and later stores it in the further memory 216.

[0058] In step S2012, the newAppKey in the further memory 216 inherits appPolicy and appID of the previous application key from the first memory 214.

[0059] Therefore, the device 208 can thus be configured to execute the application with the new application key, where such key is securely installed in the device 208 utilising a significantly lower amount of bandwidth, when compared to

securely sending asymmetric keys associated with an application policy each time a device need to execute an application managed by the application owner.

[0060] A preferred aspect of the present disclosure involves the combination of all embodiments described above. Therefore, this includes an example where the application policy, first or root key, and application ID is installed in the device as described in the first, third and fifth aspects, in association with the first embodiment of FIG. 1; followed by installation of the further application key or new application key on the device based on the second, fourth and sixth aspects associated with the second embodiment of FIG. 2.

[0061] As a result, less or reduced or lower bandwidth intensive means of secure application key transfer to a remote device is possible, by making through key derivation functions that generate key seeds and transport keys from application credentials that can be regenerated at the remote device for future use. New application keys may then be transferred to the device for installation without application policies and/or identifiers as these can be inherited from the root application key, which therefore reduces the overall bandwidth usage while providing for a robust, scalable, seamless and highly secure key installation technique.

[0062] FIG. 3, illustrates a block diagram of one implementation of a computing device 300 within which a set of instructions, for causing the computing device to perform any one or more of the methodologies discussed herein, may be executed. In alternative implementations, the computing device may be connected (e.g., networked) to other machines in a Local Area Network (LAN), an intranet, an extranet, or the Internet. The computing device may operate in the capacity of a server or a client machine in a client-server network environment, or as a peer machine in a peer-to-peer (or distributed) network environment. The computing device may be a personal computer (PC), a tablet computer, a set-top box (STB), a Personal Digital Assistant (PDA), a cellular telephone, a web appliance, a server, a network router, switch or bridge, or any machine capable of executing a set of instructions (sequential or otherwise) that specify actions to be taken by that machine.

[0063] Further, while only a single computing device is illustrated, the term “computing device” shall also be taken to include any collection of machines (e.g., computers) that individually or jointly execute a set (or multiple sets) of instructions to perform any one or more of the methodologies discussed herein.

[0064] The example computing device 300 includes a processing device 302, a main memory 304 (e.g., read-only memory (ROM), flash memory, dynamic random access memory (DRAM) such as synchronous DRAM (SDRAM) or Rambus DRAM (RDRAM), etc.), a static memory 306 (e.g., flash memory, static random access memory (SRAM), etc.), and a secondary memory (e.g., a data storage device 318), which communicate with each other via a bus 330.

[0065] Processing device 302 represents one or more general-purpose processors such as a microprocessor, central processing unit, or the like. More particularly, the processing device 302 may be a complex instruction set computing (CISC) microprocessor, reduced instruction set computing (RISC) microprocessor, very long instruction word (VLIW) microprocessor, processor implementing other instruction sets, or processors implementing a combination of instruction sets. Processing device 302 may also be

one or more special-purpose processing devices such as an application specific integrated circuit (ASIC), a field programmable gate array (FPGA), a digital signal processor (DSP), network processor, or the like. Processing device **302** is configured to execute the processing logic (instructions **322**) for performing the operations and steps discussed herein.

[0066] The computing device **300** may further include a network interface device **308**. The computing device **300** also may include a video display unit **310** (e.g., a liquid crystal display (LCD) or a cathode ray tube (CRT)), an alphanumeric input device **312** (e.g., a keyboard or touchscreen), a cursor control device **314** (e.g., a mouse or touchscreen), and an audio device **316** (e.g., a speaker).

[0067] The data storage device **318** may include one or more machine-readable storage media (or more specifically one or more non-transitory computer-readable storage media) **328** on which is stored one or more sets of instructions **322** embodying any one or more of the methodologies or functions described herein. The instructions **322** may also reside, completely or at least partially, within the main memory **304** and/or within the processing device **302** during execution thereof by the computer system **300**, the main memory **304** and the processing device **302** also constituting computer-readable storage media.

[0068] The various methods described above may be implemented by a computer program. The computer program may include computer code arranged to instruct a computer to perform the functions of one or more of the various methods described above. The computer program and/or the code for performing such methods may be provided to an apparatus, such as a computer, on one or more computer readable media or, more generally, a computer program product. The computer readable media may be transitory or non-transitory. The one or more computer readable media could be, for example, an electronic, magnetic, optical, electromagnetic, infrared, or semiconductor system, or a propagation medium for data transmission, for example for downloading the code over the Internet. Alternatively, the one or more computer readable media could take the form of one or more physical computer readable media such as semiconductor or solid state memory, magnetic tape, a removable computer diskette, a random access memory (RAM), a read-only memory (ROM), a rigid magnetic disc, and an optical disk, such as a CD-ROM, CD-R/W or DVD.

[0069] In an implementation, the modules, components and other features described herein can be implemented as discrete components or integrated in the functionality of hardware components such as ASICs, FPGAs, DSPs or similar devices.

[0070] A “hardware component” is a tangible (e.g., non-transitory) physical component (e.g., a set of one or more processors) capable of performing certain operations and may be configured or arranged in a certain physical manner. A hardware component may include dedicated circuitry or logic that is permanently configured to perform certain operations. A hardware component may be or include a special-purpose processor, such as a field programmable gate array (FPGA) or an ASIC. A hardware component may also include programmable logic or circuitry that is temporarily configured by software to perform certain operations.

[0071] Accordingly, the phrase “hardware component” should be understood to encompass a tangible entity that

may be physically constructed, permanently configured (e.g., hardwired), or temporarily configured (e.g., programmed) to operate in a certain manner or to perform certain operations described herein.

[0072] In addition, the modules and components can be implemented as firmware or functional circuitry within hardware devices. Further, the modules and components can be implemented in any combination of hardware devices and software components, or only in software (e.g., code stored or otherwise embodied in a machine-readable medium or in a transmission medium).

[0073] Unless specifically stated otherwise, as apparent from the following discussion, it is appreciated that throughout the description, discussions utilizing terms such as “receiving”, “determining”, “obtaining”, “sending”, “implementing”, or the like, refer to the actions and processes of a computer system, or similar electronic computing device, that manipulates and transforms data represented as physical (electronic) quantities within the computer system’s registers and memories into other data similarly represented as physical quantities within the computer system memories or registers or other such information storage, transmission or display devices.

[0074] It is to be understood that the above description is intended to be illustrative, and not restrictive. Many other implementations will be apparent to those of skill in the art upon reading and understanding the above description. Although the present disclosure has been described with reference to specific example implementations, it will be recognized that the disclosure is not limited to the implementations described, but can be practiced with modification and alteration within the spirit and scope of the appended claims. Accordingly, the specification and drawings are to be regarded in an illustrative sense rather than a restrictive sense. The scope of the disclosure should, therefore, be determined with reference to the appended claims, along with the full scope of equivalents to which such claims are entitled.

1. A method for securing content to a device, comprising: receiving, at the device, from an application owner in a secure network environment, an encrypted application root key and data comprising an application policy and content for a respective application, wherein the respective application is installed on the device; and periodically installing, at the device, subsequent application keys derived from a key ladder.
2. The method according to claim 1, wherein the application root key is a symmetric key and wherein the subsequent application keys are symmetric keys.
3. The method according to claim 1, wherein the application root key is an asymmetric key and wherein the subsequent application keys are symmetric keys.
4. The method according to claim 1, wherein the application root key is a symmetric key and wherein the subsequent application keys are asymmetric keys.
5. The method according to claim 1, wherein the application root key is an asymmetric key and wherein the subsequent application keys are asymmetric keys.
6. The method according to claim 1, wherein the subsequent application keys are generated by the application owner with the key ladder.

7. The method according to claim 6, further comprising: generating, at the device, a key seed using the application policy and an application identifier for the respective application; and sending the key seed to the application owner, wherein the application owner generates a transport key using the key seed and the application root key.
8. The method according to claim 7, wherein the application owner encrypts the subsequent application keys using the transport key and sends the encrypted subsequent application keys to the device for installation.
9. The method according to claim 1, wherein the subsequent application keys derived from the key ladder are configured to inherit the application policy associated with the received encrypted application root key, for further usage.
10. The method according to claim 1, wherein the encrypted application root key is encrypted with a symmetric device encryption key.
11. The method according to claim 10, further comprising decrypting, at the device, the encrypted application root key with a symmetric device encryption key.
12. The method according to claim 1, wherein the encrypted application root key is encrypted with a public device encryption key.
13. The method according to claim 12, further comprising decrypting, at the device, the encrypted application root key with a private device encryption key.
14. The method according to claim 1, wherein the encrypted application root key is signed with a symmetric device signature key.
15. The method according to claim 14, further comprising verifying, at the device, a signature of the encrypted application root key with a symmetric device signature key.

16. The method according to claim 1, wherein the encrypted application root key is signed with a private device signature key.

17. The method according to claim 16, further comprising verifying, at the device, a signature of the encrypted application root key with a public device signature key.

18. The method according to claim 1, wherein the application policy regulates data transfer between the device and the application owner.

19. A device for executing an application using one or more application keys, the device comprising:

memory, in which the application is installed; and

one or more processors configured to

receive from an application owner in a secure network environment, an encrypted application root key and data comprising an application policy and content for the application; and

periodically install subsequent application keys derived from a key ladder.

20. A non-transitory computer-readable storage medium including computer executable instructions, wherein the instructions, when executed by one or more processors of a computing device, cause the device to perform a method for securing content to a device, comprising:

receiving, at the device, from an application owner in a secure network environment, an encrypted application root key and data comprising an application policy and content for a respective application, wherein the respective application is installed on the device; and periodically installing, at the device, subsequent application keys derived from a key ladder.

* * * * *